Rapport Labo 1

Kenan Augsburger

22 September 2025

Contents

1. Installation and Launch

? What are tokens used for and how do they look like in OpenBao?

Tokens are used to authenticate when accessing the vault. Tokens can be used directly or generated dynamically through auth methods. (Using configured authentication providers like github OAuth)

The example provided in the documentation for token formats is the following

```Plain Text
b.n6keuKu5Q6pXhaIcfnC9cFNd
r.JaKnR2AIHNk3fC4SGyyyDVoQ9O
s.raPGTZdARXdY0KvHcWSpp5wWZIHNT
```

Where each token is prefixed according to their type.

- `s.` : Service tokens
- `b.` : Batch tokens
- `r.` : Recovery tokens

A token's body is a random `Base62` value of 24 characters.

Tokens documentation°

i Note

Root tokens are tokens with the `root` policy attached. Those tokens provide access to anything within OpenBao and can be set to never expire without any renewal required.

Those tokens are hard to create and should only be used for the initial setup or emergencies. They should be revoked otherwise.

1. The initial root token is generated at `bao operator init` time and has no expiration
2. Root tokens can create other root tokens. Tokens with an expiration cannot create a token without expiration.
3. Using `bao operator generate-root` with permission of a quorum of unseal key holders. (Similar to the key ceremony I guess)

i Environment setup

```Fish
docker compose up -d
set -x BAO_ADDR http://localhost:8200
# Login prompt, use a token when prompted
bao login
bao status
```

? How do you set up what are the capabilities of a token?

Capabilities are a property of policies. With policies you can define what can be done based on a path. Once you have a policy you can apply the policy to a token.

For example, you can create the policy `Foo`.

```terraform
# file: foo.hcl
# Allow read of anything within secret/foo
path "secret/foo/*" {
  capabilities = ["read"]
}

# Allow read and update for secret/bar
path "secret/bar" {
  capabilities = ["read", "update"]
}
```

```fish
bao policy write foo foo.hcl
```

And create a token with the new policy

```fish
bao token create -display-name foo -policy foo
# Key                  Value
# ---                  -----
# token                s.GHmDKZEH0TKGJQ6I5McAvoy9
# token_accessor       v0EJuFUt81Hy6J7q3YE74xJh
# token_duration       768h
# token_renewable      true
# token_policies       ["default" "foo"]
# identity_policies    []
# policies             ["default" "foo"]
```

Policy syntax°

? What is sealing and unsealing and what is it used for?

Sealing means that the server will throw away the root key it was using to decipher the secrets. This means that an intruder cannot extract the key from a memory dump but it also means that the service won't be available anymore.

Unsealing is the process of deciphering the root key. This requires the use of unsealing keys which are shared between multiple people using Shamir's secret sharing. (Similar to the key ceremony for the root CAs.)

As stated, the goal of sealing is to ensure that secrets won't leak if signs of intrusion are detected.

2. Main App Description

> **? Why isn't it a great idea to store the ciphertexts on Vault? (We do it to avoid having to setup a database).**
>
> The vault's goal is to be a place to securely store secrets. As such it's optimized for security before performances. Accessing each message and metadata in the vault is done through the http(s) api and it doesn't seem like you can read in batch. This means that one message results in one http call (which will in turn slow down the vault as the number users/messages grows).
>
> And more importantly, each message is stored as a ciphertext which means it's already encrypted. Storing it in the vault means that there are two layers of encryption which increases processing time without any meaningful security gains.

> **i Quick setup**
>
> I created a `fish` script to quickly setup the dev server with the kv and transit engines
>
> ```fish
> ./setup.fish
> ```
> Fish

3. Creating Groups